

STUDENT TECHNICAL REPORT / SUBMISSION RECORD

Amankos Danial BDA-2405

Assignment 1. Python Fundamentals for Data Streaming

AITU Campus Shuttle and Mobility Data

This report records the implementation and execution evidence for the supplied synthetic campus shuttle dataset. The code files named in each section are included in the same submission folder.

1. Student and Submission Information

Required field	Student entry
Student full name	Amankos Danial
Student ID	240260
Group / cohort	BDA-2405
Instructor	Serek Azamat
Submission date	10.09.2026
Git repository link	https://github.com/subdragon34/Python-Fundamentals-for-Data-Streaming-Project
Python version	3.10.11

2. Work Record

Item	Student entry
Work start date	10 September 2026
Work completion date	10 September 2026
Main tools / libraries used	Python standard library, FastAPI, Uvicorn, pytest, HTTPX
Source-code file names	main.py, api.py, export_json.py
Test file / notebook name	test_assignment.py
README location	README.md in the project root

3. Report Structure

3.1 Data Structures and Parsing

I represented the supplied shuttle data as a list of dictionaries. Each dictionary contains six fields from one event: Timestamp, Route, Bus, Passengers, Speed_kmh, and Status. I chose dictionaries because they are easy to read in Python and their structure also matches the JSON objects used later by the API.

The parsing functions are kept separate from the main validation function. `parse_timestamp()` converts an HH:MM string to `datetime.time`, `parse_passengers()` returns a non-negative integer, and `parse_speed()` returns a numeric speed. `validate_event()` combines these checks and returns a cleaned event record.

Required evidence	Student response / reference
Explanation	List of dictionaries with separate parsing functions and one shared validation function.
Code / file / function reference	main.py - <code>EVENTS</code> , <code>parse_timestamp()</code> , <code>parse_passengers()</code> , <code>parse_speed()</code> , <code>validate_event()</code>
Execution evidence / result	evidence/main_output.txt shows all ten supplied records after validation.
Student-specific observation or decision	I used dictionaries instead of a DataFrame because the same event shape is used for JSON and FastAPI.

3.2 Event Validation

The validator first checks that all required fields are present. Timestamp must be a valid HH:MM time. Passengers must be a non-negative integer. Speed_kmh must be between 0 and 120 inclusive. Status must be exactly ON_ROUTE or STOPPED. I also reject an empty Route or Bus value. Validation errors are collected so the API can return more than one problem in a single response.

Field	Python type	Validation rule	Example
Timestamp	datetime.time	Valid HH:MM time	08:00
Passengers	int	Non-negative integer	18
Speed_kmh	int or float	0–120 inclusive	31
Status	str	ON_ROUTE or STOPPED	ON_ROUTE
Required evidence		Student response / reference	
Explanation		Required-field checks plus assignment rules for timestamp, passengers, speed, and status.	
Code / file / function reference		main.py — EventValidationError and validate_event()	
Execution evidence / result		evidence/pytest_output.txt and evidence/api_invalid_response.json	
Student-specific observation or decision		I collect all field errors so one invalid API request can show several problems at once.	

3.3 Streaming Generator

The `event_generator()` function yields one validated event at a time. It keeps only the previous timestamp so it can check that the supplied events remain chronological. The function does not create a second list of processed records.

This behavior is useful for streaming because the caller can finish processing the current event before asking the generator for the next one. For an unlimited stream, memory use does not need to grow with the total number of past events.

Required evidence	Student response / reference
Explanation	Generator validates and yields each record while keeping only the previous timestamp.
Code / file / function reference	main.py — event_generator()
Execution evidence / result	evidence/main_output.txt; test_generator_yields_all_events_in_order in test_assignment.py
Student-specific observation or decision	I check chronological order without sorting the whole source inside the processor.

3.4 Streaming Simulation

`process_stream()` receives validated events from `event_generator()` and processes them one at a time. Valid records are marked as processed. An invalid record would be marked as not processed and its validation reason would be returned.

Event	Passengers	Speed	Status	Processed?	Reason
1	18	31	ON_ROUTE	Yes	Valid event
2	22	28	ON_ROUTE	Yes	Valid event
3	21	29	ON_ROUTE	Yes	Valid event
4	25	27	ON_ROUTE	Yes	Valid event
5	24	0	STOPPED	Yes	Valid event

Event 5 is accepted even though its speed is 0 km/h. Zero is inside the allowed speed range and the STOPPED status correctly describes the event.

Required evidence	Student response / reference
Explanation	Each input record is handled separately and receives a processed flag and reason.
Code / file / function reference	main.py — process_stream()
Execution evidence / result	evidence/main_output.txt contains the processing result for all ten events.
Student-specific observation or decision	A stopped bus is not treated as invalid when speed is 0 and status is STOPPED.

3.5 JSON and FastAPI Endpoint

The supplied events are stored in `events.json` using the same field names as the Python dictionaries. The FastAPI application defines `POST /events`, which accepts one event as a JSON object. The endpoint reuses `validate\_event()` instead of maintaining a second set of validation rules.

For an accepted event, the endpoint returns `accepted: true`, an empty validation-error list, and the calculated occupancy category. For a rejected event, it returns `accepted: false`, the collected validation errors, and a null occupancy category.

Passenger count	Occupancy category
0–10	LOW
11–20	MEDIUM
21–30	HIGH
>30	OVER CAPACITY
Required evidence	Student response / reference
Explanation	JSON export plus POST /events with accepted/rejected response, errors, and occupancy.
Code / file / function reference	api.py — receive_event(); main.py — occupancy_category(); events.json
Execution evidence / result	evidence/api_valid_response.json and evidence/api_invalid_response.json
Student-specific observation or decision	The API calls the same validator as the stream so both paths apply identical rules.

POST

/events

Receive Event

Parameters

Try it out

No parameters

Request body

required

application/json

Example Value

Schema

```
{
  "additionalProp1": {}
}
```

Responses

Code	Description	Links
200	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>"string"</pre>	No links
422	Validation Error Media type application/json Example Value Schema <pre>{ "detail": [{ "loc": ["string", 0], "msg": "string", "type": "string", "input": "string", "ctx": {} }] }</pre>	No links

```
{
  "Timestamp": "08:09",
  "Route": "AITU-Campus-Residence",
  "Bus": "B02",
  "Passengers": 30,
  "Speed_kmh": 24,
  "Status": "ON_ROUTE"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/events' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Timestamp": "08:09",
    "Route": "AITU-Campus-Residence",
    "Bus": "B02",
    "Passengers": 30,
    "Speed_kmh": 24,
    "Status": "ON_ROUTE"
  }'
```

Request URL

http://127.0.0.1:8000/events

Server response

Code Details

200

Response body

```
{
  "accepted": true,
  "validation_errors": [],
  "occupancy_category": "HIGH"
}
```



Download

Response headers

```
content-length: 68
content-type: application/json
date: Thu, 18 Sep 2026 17:41:46 GMT
server: uvicorn
```

Responses

Edit Value
Schema

```

{
  "Timestamp": "08:09",
  "Route": "AITU-Campus-Residence",
  "Bus": "B02",
  "Passengers": 30,
  "Speed_kmh": -15,
  "Status": "ON_ROUTE"
}

```

Execute
Clear

Responses

Curl

```

curl -X 'POST' \
  http://127.0.0.1:8000/events \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "Timestamp": "08:09",
    "Route": "AITU-Campus-Residence",
    "Bus": "B02",
    "Passengers": 30,
    "Speed_kmh": -15,
    "Status": "ON_ROUTE"
  }'

```

Request URL

```

http://127.0.0.1:8000/events

```

Server response

Code
Details

200

Response body

```

{
  "accepted": false,
  "validation_errors": [
    "Speed_kmh must be between 0 and 120"
  ],
  "occupancy_category": null
}

```

Download

3.6 Data Analysis

The analysis runs in one pass over the event generator. It maintains a running count and passenger total for the average, tracks the largest passenger count and its event, and increments a counter whenever Status is STOPPED.

Measure	Result
Average passenger count	24.4
Maximum passenger count	30
Number of STOPPED events	1
Busiest minute	08:09
Busiest bus	B02
Required evidence	Student response / reference
Explanation	Single-pass calculation over validated events.
Code / file / function reference	main.py — analyze_events()
Execution evidence / result	evidence/main_output.txt and test_analysis_results in test_assignment.py
Student-specific observation or decision	I avoided converting the stream to a DataFrame because the assignment focuses on streaming behavior.

Validated shuttle events

```
-----
01 | 08:00 | B01 | 18 passengers | 31 km/h | ON_ROUTE
02 | 08:01 | B02 | 22 passengers | 28 km/h | ON_ROUTE
03 | 08:02 | B01 | 21 passengers | 29 km/h | ON_ROUTE
04 | 08:03 | B02 | 25 passengers | 27 km/h | ON_ROUTE
05 | 08:04 | B01 | 24 passengers | 0 km/h | STOPPED
06 | 08:05 | B02 | 26 passengers | 30 km/h | ON_ROUTE
07 | 08:06 | B01 | 23 passengers | 32 km/h | ON_ROUTE
08 | 08:07 | B02 | 28 passengers | 26 km/h | ON_ROUTE
09 | 08:08 | B01 | 27 passengers | 25 km/h | ON_ROUTE
10 | 08:09 | B02 | 30 passengers | 24 km/h | ON_ROUTE
```

Streaming simulation

```
-----
Event 01 | passengers=18 | speed=31 | status=ON_ROUTE | processed=Yes | reason=Valid event
Event 02 | passengers=22 | speed=28 | status=ON_ROUTE | processed=Yes | reason=Valid event
Event 03 | passengers=21 | speed=29 | status=ON_ROUTE | processed=Yes | reason=Valid event
Event 04 | passengers=25 | speed=27 | status=ON_ROUTE | processed=Yes | reason=Valid event
Event 05 | passengers=24 | speed=0 | status=STOPPED | processed=Yes | reason=Valid event
Event 06 | passengers=26 | speed=30 | status=ON_ROUTE | processed=Yes | reason=Valid event
Event 07 | passengers=23 | speed=32 | status=ON_ROUTE | processed=Yes | reason=Valid event
Event 08 | passengers=28 | speed=26 | status=ON_ROUTE | processed=Yes | reason=Valid event
Event 09 | passengers=27 | speed=25 | status=ON_ROUTE | processed=Yes | reason=Valid event
Event 10 | passengers=30 | speed=24 | status=ON_ROUTE | processed=Yes | reason=Valid event
```

Analysis

```
-----
Average passenger count: 24.4
Maximum passenger count: 30
STOPPED events: 1
Busiest minute: 08:09
Busiest bus: B02
```

3.7 Testing and Code Quality

The project contains automated tests for valid and invalid events, boundary behavior, generator order, analysis results, and both successful and rejected API requests. The file contains more than the minimum three tests so the main rules and edge cases can be checked independently.

Test case	Expected result	Actual result	Status
Valid event	Accepted	Accepted	PASS
Negative passengers	Rejected	Rejected	PASS
Speed 121 km/h	Rejected	Rejected	PASS
Status MOVING	Rejected	Rejected	PASS
Time 25:61	Rejected	Rejected	PASS
12.5 passengers	Rejected	Rejected	PASS
Occupancy boundaries	Correct four categories	Correct	PASS
Supplied-data analysis	24.4 / 30 / 1 / 08:09 / B02	Matched	PASS
Valid API request	accepted=true, HIGH	Matched	PASS
Invalid API request	4 validation errors	Matched	PASS
Required evidence		Student response / reference	
Explanation		Automated tests cover parsing/validation, stream order, occupancy, analysis, and API behavior.	
Code / file / function reference		test_assignment.py	
Execution evidence / result		evidence/pytest_output.txt	
Student-specific observation or decision		My additional test rejects a decimal passenger count because passenger count represents whole people.	

```
C:\Users\ania\Desktop\RTDADM\Assignment1_Dania>python -m pytest -q
..... [100%]
===== warnings summary =====
...\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.10_qbz5n2kfra8p0\LocalCache\local-packages\Python310\site-packages\starlette\testclient.py:45
C:\Users\ania\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.10_qbz5n2kfra8p0\LocalCache\local-packages\Python310\site-packages\starlette\testclient.py:45: DeprecationWarning: The anyio.abc.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
  _PortalFactoryType = Callable[[], AbstractContextManager[anyio.abc.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
11 passed, 1 warning in 1.46s
```

3.8 Technical Interpretation

Generators are preferable for an unlimited stream because they produce values lazily. A normal approach that repeatedly loads all past events would require more memory as the stream grows. With a generator, the next event is requested only when the current one is ready to be processed.

One limitation is that this assignment uses a fixed in-memory educational dataset rather than a live data source. A realistic improvement would be to read events from a queue such as Kafka and store accepted results in a database while keeping the same validation functions.

Required evidence	Student response / reference
Explanation	Lazy event handling keeps memory use bounded for long-running streams.
Code / file / function reference	main.py - event_generator(), process_stream(), analyze_events()
Execution evidence / result	evidence/main_output.txt shows sequential handling of the supplied records.
Student-specific observation or decision	The current source is intentionally simple; a message broker would be the next realistic extension.

4. Individual Work and Anti-Plagiarism Record

The implementation references exact functions and output files so the reported results can be reproduced during the oral demonstration.

Individual evidence	Student entry
My unique implementation/design decision	I use one shared validate_event() function for both streaming logic and the FastAPI endpoint, and it can collect several field errors in one request.
My own test case	A passenger count of 12.5 is rejected because passenger counts must be whole non-negative integers.
Bug/problem I encountered and solution	Validated timestamps become datetime.time objects, which the standard JSON encoder cannot serialize directly. I kept external JSON timestamps as HH:MM strings and use format_event() when a validated event needs a JSON-safe form.
Exact code/repository/commit reference	main.py: validate_event(), event_generator(), process_stream(), occupancy_category(), analyze_events(); api.py: receive_event(). Add Git commit reference after uploading.
One limitation of my implementation	The source is a fixed in-memory synthetic dataset instead of a live continuous source.
One improvement I would make	Connect the processor to Kafka or another queue and persist accepted events in a database.

5. Submission Package Checklist

- ☒ Python source code
- ☒ Technical report
- ☒ JSON sample

- ☒ FastAPI endpoint code
- ☒ Completed assignment tables
- ☒ At least 3 test cases
- ☒ README with run instructions
- ☒ Execution evidence / API responses
- ☐ Oral defense yet to be completed

6. Student Declaration of Originality

I confirm that this submission represents my own work. I have not copied another student's source code, calculations, tables, screenshots, test results, API responses, or written explanations. Any external material or reused code has been identified. I understand that the instructor may request a live demonstration, code modification, or explanation of any submitted component.

Role	Name	Signature	Date
Student	Amankos Danial Dauletuly		10.09.2026

Appendix A — Execution Evidence

The following files are included in the evidence folder and were generated by running the submitted code:

evidence/main_output.txt — validated events, streaming simulation, and analysis

evidence/pytest_output.txt — automated test result

evidence/api_output.txt — valid and invalid endpoint responses

evidence/api_valid_response.json — accepted request example

evidence/api_invalid_response.json — rejected request with multiple validation errors

evidence/json_export_output.txt — JSON export confirmation

evidence/debugging_note.txt — timestamp serialization issue and solution